GenAI Milestones 4 and 5

Sarthak Jain | 23f3000839 | September 2026 term

These are my worked answers for the two milestone forms. I have kept the calculations and the reasons for the decisions here; the lines marked Form entry follow the shorter format requested in each question. Training results come from the separate Milestone 4 experiment, rather than from the larger final-project runs.

Milestone 4 Training improvements and diagnosis

I selected the first 1,200 unique image IDs in ASCII order and retained all four class rows for each image. The first ID is 000HAM628W and the last is 3V4QMBN61P. Masks use the native 128 x 800 canvas, 1-based RLE starts and column-major decoding, with overlong runs clipped at the canvas boundary.

There is one small problem with the prescribed split: defect_a+defect_b+defect_c occurs only once. Relabelling it rare_other still leaves a singleton, which scikit-learn cannot stratify. I kept image 30YZYD878X in training and stratified the other 1,199 images with seed 42, reserving 240 validation images. This gives 960 training images and 240 validation images. Every experiment uses that same saved split.

Question 1 Image and class imbalance

I counted a class as present when its stripped RLE string was nonempty. There are 578 completely clean images. The positive image counts are 107 for defect_a, 35 for defect_b, 438 for defect_c and 181 for defect_d. These add up to 761 positive image-class pairs, rather than 761 different images, because an image can contain several defect classes.

Before merging rare combinations, there are 15 distinct combination strings. The clean group is the largest, with 578 images. The smallest group is defect_a+defect_b+defect_c, with one image. Defect_b is rare at the image level, but the pixel-level calculation in Question 2 shows a much stronger imbalance.

Form entry: 578; 107, 35, 438, 181; 15; 578, 1

Question 2 Pixel imbalance and a candidate positive weight

Across the 1,200 defect_b masks, I decoded 19,654 positive pixels out of 122,880,000 possible pixels. That is 0.0160%. Among the 35 defect_b-positive images, the average annotated defect_b area is 19,654 / 35 = 561.5 pixels.

There are 122,860,346 negative pixels. Dividing negatives by positives gives 6,251.16, so the rounded candidate BCE pos_weight is 6,251. This number describes how sparse the target is; I did not insert it into the controlled training runs. Such a large loss weight would be a separate change and could make a few labelled or noisy pixels dominate the gradients.

Form entry: 0.0160%, 561.5, 6251

Question 3 Bounded square-root sampling

I calculated each image weight from its unmerged combination group, using the smaller of sqrt(578 / group_size) and 5. The total probability mass for a group is its image count multiplied by that weight. Dividing each group mass by the sum of all group masses gives its expected share of draws.

Clean images account for 48.17% of natural draws, compared with 28.81% under this sampler. The expected number of defect_b-positive draws rises from 35.00 to 104.69 in 1,200 replacement draws: an enrichment of 2.99 times. Ten combination groups reach the weight cap. These are expectations, not a guarantee that each sampled epoch contains those exact counts. For R2, I recomputed the group sizes on the 960 training images only, as Question 10 requires.

Form entry: 28.81%, 48.17%; 104.69, 35.00; 2.99x; 10

Question 4 Warmup, cosine decay and scheduler cadence

I applied the given multiplier to the base learning rate of 3e-4. At step 15, the multiplier is 15 / 60, so the learning rate is 7.50e-5. Step 270 is halfway through the 420-step cosine portion, giving 1.50e-4. At step 400, the same expression gives 2.61e-5.

Calling this step-based scheduler once per epoch advances it only eight steps in eight epochs. Its learning rate is then 3e-4 x 8 / 60 = 4.00e-5, so it is still near the beginning of warmup. I would expect slow learning rather than the intended rise and decay. In the scheduled training run, I stepped after every optimizer update. I retained the question's 480-step reference schedule for the requested six-epoch budget, so that run ends at step 360 of the supplied schedule.

Form entry: 7.50e-05, 1.50e-04, 2.61e-05; 4.00e-05; The learning rate stays in the early warmup range, so learning is slow and the loss curves may look flat or underfit.

Question 5 Early stopping and checkpoint rollback

With min_delta = 0.002, epoch 7 is an improvement: its score is 0.671. Epochs 8, 9 and 10 then fail the strict test score > best + min_delta. Although 0.672 at epoch 9 exceeds 0.671, it does not exceed 0.673, so the patience counter does not reset. The counter reaches three at epoch 10, and the saved epoch 7 checkpoint is restored.

With min_delta = 0, epoch 9 becomes an improvement, and epoch 11 improves again to 0.673. There is no stretch of three failures after either of those improvements within the twelve supplied epochs, so training does not stop. The best checkpoint at the end belongs to epoch 11.

Form entry: 10, 7, 0.671; No, 11

Question 6 AMP memory and gradient accumulation

The parameter-related storage consists of four FP32 arrays: weights, gradients and two AdamW moment buffers. Each needs 25 million x 4 bytes = 100 MB, giving 400 MB in total. AMP only halves the activation term under this question's assumptions; it does not halve those four arrays.

At batch size 16, FP32 needs 400 + 16 x 150 = 2,800 MB. AMP needs 400 + 16 x 75 = 1,600 MB. With a 3,400 MB budget, the largest batches are 20 and 40 respectively. These calculations deliberately exclude the overheads that the question asks us to ignore.

For accumulation, a micro-batch of 8 repeated over 4 backward passes gives an effective batch size of 32. The 960 training images give 120 micro-batches and 30 optimizer updates per epoch. Each

micro-batch loss is divided by 4 before backward. A two-epoch warmup therefore takes 60 optimizer updates, not 240 micro-batches.

Form entry: 400; 2800, 1600; 20, 40; 32; 120, 30; 4; 60

Question 7 Why naive Dice rewards an empty prediction

The subset has 4,800 image-class pairs, of which 4,039 are empty. An all-empty predictor gets Dice 1 on those pairs and Dice 0 on the 761 positive pairs. Its naive mean Dice is therefore 4,039 / 4,800 = 0.8415. Its official score is only 0.3000: foreground Dice is zero, while FalsePositiveScore and EmptyAccuracy are both one.

In the toy validation set, 306 of 340 empty pairs are correctly empty. Each of the other 34 has a 1,024-pixel blob, which is 1% of a mask. The mean false-positive pixel fraction over empty pairs is 0.001, giving FalsePositiveScore 0.999. EmptyAccuracy is 0.9. With ForegroundDice 0.5, the official score is 0.6398.

The same model's naive Dice is (60 x 0.5 + 306) / 400 = 0.8400. The all-empty baseline gets 340 / 400 = 0.8500 on this set. Naive Dice therefore ranks the useful model below the baseline, which is why I monitor the official score and its components.

Form entry: 4039; 0.8415; 0.3000; 0.6398, 0.8400; naive Dice, 0.8500

Question 8 Training curves and checkpoint choice

I used the RGB SmallUNet from Milestone 2, AdamW at 3e-4, batch size 16, AMP, seed 42, native image size and a probability threshold of 0.5. The loss and joint image/mask augmentations stayed the same as the baseline. This run used the warmup and cosine schedule. It is also R1 in Question 10 because those are exactly the same settings.

The lowest validation loss occurs at epoch 6. The highest official validation score is 0.300, at epoch 4. At epoch 6, training loss is 0.4946 and validation loss is 0.4922, so validation minus training loss is -0.002. These are different criteria: a smaller loss does not necessarily produce the best thresholded masks or the best official score.

I would monitor the official validation score and keep epoch 4, the earliest tied best. Both losses fall, but foreground Dice stays zero. The run is underfitting the defects and mostly learning background, rather than showing evidence of overfitting. Split note: rare_other has one image, so I kept it in training and stratified the remainder to obtain 960/240.

Question 9 Defect sizes and missed-pair counts

After decoding all positive masks in the subset, I found 5 pairs with 1–100 positive pixels, 99 with 101–500 pixels and 657 with at least 501 pixels. The largest bin contains 3,953,580 of the 3,983,874 annotated positive pixels, or 99.24%.

Applying the hypothetical presence recalls gives expected misses of 5 x 0.70 = 3.50, 99 x 0.30 = 29.70 and 657 x 0.05 = 32.85. The total is 66.05 missed positive pairs, with the 501+ bin contributing the most. Its recall is highest, but there are far more pairs in that bin. Pair counts and pixel counts answer different questions: one missed large defect can outweigh many tiny defects in lost pixels.

Form entry: 5, 99, 657; 99.24%; 66.05, 501+ px; Missing a few large defects can lose more pixels, while missing many tiny defects can dominate the number of missed pairs.

Question 10 Controlled changes and seed noise

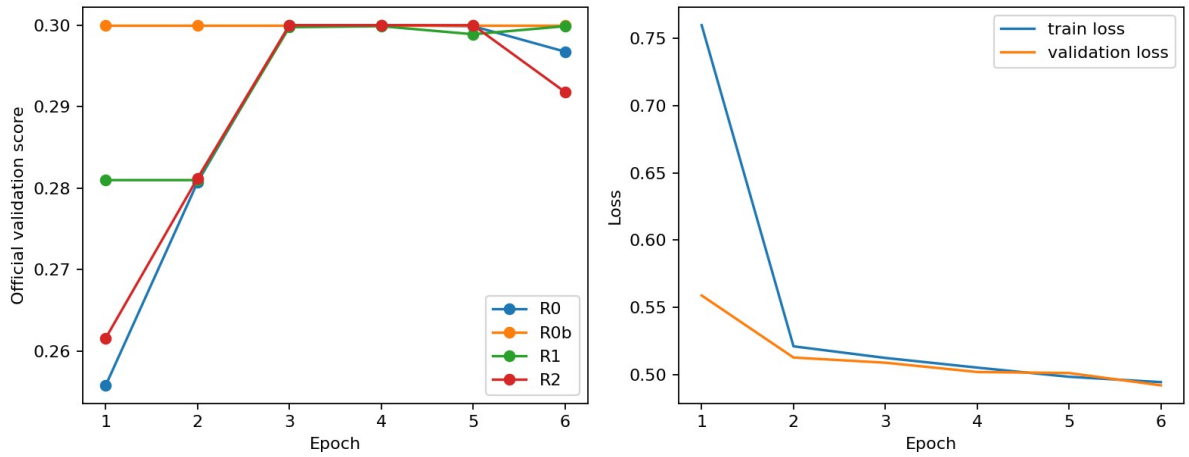
All four runs use six epochs and the same saved split. R0 uses seed 42, a constant learning rate and natural sampling. R0b changes only the seed to 7. R1 adds the scheduler to R0. R2 adds the bounded sampler to R0, using training-only combination counts. AMP stays on throughout. I restored each run's best official-score checkpoint before measuring its empty-pair false-alarm rate.

Run	Best epoch	Official score	Empty-pair false alarms
R0	3	0.3000	0.0%
R0b	1	0.3000	0.0%
R1	4	0.2999	0.1%
R2	3	0.3000	0.0%

The seed-noise estimate is |R0 - R0b| = 0.0000. The R1 gain over R0 is -0.0001, and the R2 gain is +0.0000. Against the required threshold of twice the noise (0.0000), the scheduler does not qualify as an improvement and the sampler does not. I compared the unrounded values, since rounding first could change a borderline decision. Two seeds are only a rough noise check, so this should not be read as a formal significance test.

Changing the sampler, scheduler and AMP together would make the source of a gain unclear. The separate runs show which change produced each observed difference. The sampler deliberately makes defects more frequent in training. Its change in false alarms on naturally distributed empty validation pairs shows the practical effect of that changed training prior; it is not a measurement of posterior calibration by itself.

Figure 1. Six-epoch validation scores and R1 loss curves.



Milestone 5 Post-processing and inference

Questions 1–4 and 6–8 use the numerical examples supplied in the form. Question 5 uses every positive pair in the full train.csv. I used strict p > T thresholds, 8-connected components where requested, and decimal GB throughout.

Question 1 Logits and the threshold boundary

A probability threshold of 0.3 corresponds to log(0.3 / 0.7) = -0.8473 in logit space. At a threshold of 0.5, strict comparison selects only logits greater than zero: pixels 5 and 6. The non-strict comparison also selects pixel 4 because its logit is exactly zero and its probability is exactly 0.5.

At T = 0.3, the predicted positive pixels are 3, 4, 5 and 6. Three overlap the four true positive pixels, giving Dice 2 x 3 / (4 + 4) = 0.7500. At T = 0.5 with strict comparison, only one of the two predicted pixels overlaps the ground truth, giving Dice 2 x 1 / (2 + 4) = 0.3333.

Form entry: -0.8473, 2, 3, 0.7500, 0.3333

Question 2 Choosing a post-processing configuration

I applied the official weights to each row of the sweep table. X scores 0.70 x 0.62 + 0.20 x 0.9950 + 0.10 x 0.40 = 0.6730. Y scores 0.6908 and Z scores 0.6120. Y is the best of the three.

X has the highest foreground Dice, but Y has both a higher FalsePositiveScore and a much higher EmptyAccuracy. Those gains more than offset Y's lower foreground Dice. Raising the threshold further to Z removes more false positives, but loses too much foreground overlap. This is why I choose settings using the complete official score rather than one component.

Form entry: 0.6730, 0.6908, 0.6120, Y, FalsePositiveScore and EmptyAccuracy

Question 3 Connectivity changes what gets filtered

Under 8-connectivity, diagonal contacts join the main group into one 11-pixel component. The other two components each have one pixel, so there are three components in total. Under 4-connectivity, those diagonal bridges do not connect, giving six components.

With 8-connectivity, discarding areas below 5 leaves the 11-pixel component. Raising the cutoff to 6 gives the same result because there is no component of area 5. Both filtered masks therefore contain 11 foreground pixels. I checked component areas directly rather than treating the whole mask's area as one component.

Form entry: 3, 6, 11, 11

Question 4 A score can improve while foreground Dice falls

Before filtering, the positive-pair foreground Dice is (12 x 0.70 + 8 x 0.45) / 20 = 0.6000. The 20 small speckles and ten large blobs contribute 20,400 false-positive pixels over 80 empty pairs. FalsePositiveScore is therefore 0.9975098 and EmptyAccuracy is 50 / 80 = 0.6250.

The 50-pixel area filter removes the twenty speckles and one real positive prediction. ForegroundDice drops to (12 x 0.70 + 7 x 0.45) / 20 = 0.5775. FalsePositiveScore becomes 0.9975586, rounded to 0.9976, and EmptyAccuracy rises to 70 / 80 = 0.8750. The official score rises from 0.6820 to 0.6913. Most of the benefit comes from turning noisy empty pairs into fully empty predictions, not from the very small change in false-positive pixel mass.

Form entry: 0.5775, 0.9976, 0.8750, 0.6820, 0.6913

Question 5 The recall ceiling caused by area filtering

I decoded all 7,201 positive image-class pairs in the full training CSV and labelled their 8-connected components. A pair is wiped out when every component has area below 100. A pair with a

component of exactly 100 pixels survives because the rule removes only areas strictly below the threshold.

This removes 48 positive pairs completely: 22 from defect_a, 1 from defect_b, 24 from defect_c and 1 from defect_d. Defect_c loses the most pairs. Even perfect probability maps would lose those pairs after this filter, so this is a limitation imposed by post-processing rather than by the model. The calculation uses the full dataset, unlike the fixed subset in Milestone 4.

Form entry: 48, defect_c

Question 6 Undoing flips before averaging

For BCHW tensors, the width axis is dimension 3 and the height axis is dimension 2. I therefore use dimension 3 for a horizontal flip and dimension 2 for a vertical flip. The predicted maps need the same inverse flips to put their pixels back into the original frame.

After inversion, the three values at the original top-right pixel are 0.6, 0.5 and 0.9, giving 0.6667. The full correctly aligned average is [[0.8000, 0.6667], [0.3000, 0.3333]], with two positive pixels at strict T = 0.5. Forgetting inversion gives [[0.5000, 0.6333], [0.4667, 0.5000]], with only one positive pixel. The two values equal to 0.5 do not pass the strict threshold.

Form entry: 3, 2, 0.6667, 2, 1

Question 7 Soft averaging and hard voting are different

Averaging the three probability maps and then thresholding selects p1, p3 and p5, exactly matching the ground truth, so soft-ensemble Dice is 1.0000. Thresholding each model first and taking a majority vote selects p2, p3 and p5. It misses p1 and adds p2, giving Dice 4 / 6 = 0.6667. Soft averaging retains confidence values that the hard vote discards.

At p1, weights 0.5, 0.3 and 0.2 give 0.5 x 0.90 + 0.3 x 0.40 + 0.2 x 0.45 = 0.6600. For snapshot logits +1 and -3, averaging sigmoid probabilities gives 0.3892. Averaging the logits first gives -1, whose sigmoid is 0.2689. Only probability-space averaging exceeds the strict 0.30 threshold. Sigmoid is nonlinear, so these operations cannot be interchanged.

Form entry: 1.0000, 0.6667, 0.6600, 0.3892, 0.2689, prob_avg

Question 8 Memory and time budgets

One full set of predictions contains 5,000 x 4 x 128 x 800 = 2,048,000,000 values. This needs 8.192 GB as float32, 4.096 GB as float16 or 2.048 GB as uint8. Config A stores nine model/view sets if implemented naively, requiring 73.728 GB. Config B stores four sets, requiring 32.768 GB. Neither fits the stated 16 GB budget.

Keeping one running float32 accumulator reduces the full-set storage to 8.192 GB; a batchwise accumulator can reduce the working set further. Config A makes 45,000 forward passes and takes 9.0 minutes of pure model time. Config B makes 20,000 passes and takes 4.0 minutes.

After reserving eight of the thirty minutes for other work, 22 minutes remain. One model costs three minutes with three views, so at most seven models fit. One model costs two minutes with two views, so the mathematical maximum is eleven. Eleven uses the entire stated budget, leaving no margin for runtime variability; I would use fewer in a real submission notebook.

Form entry: 8.192, 4.096, 2.048, 73.728, 32.768, neither, 8.192, 45000, 20000, 9.0, 4.0, 7, 11

Question 9 Final inference notebook

- I would keep the native 128 x 800 image geometry and apply exactly the preprocessing used to train each checkpoint. I would put every model in evaluation mode and use inference_mode, with AMP where it is supported. Changing normalization or channel order at inference would feed a different input distribution to the model.
- For each image batch and each checkpoint, I would run the original image and the chosen flips. On BCHW tensors, horizontal flip uses dimension 3 and vertical flip uses dimension 2. I would undo each flip on its predicted probability map before accumulating it. Otherwise, the average would mix different physical pixels and blur or move defects.
- I would average the aligned probabilities over views, then combine models with fixed weights that sum to one. Thresholding comes after those averages: an early hard threshold throws away confidence information and changes a soft ensemble into a voting scheme. I would then apply each class's chosen threshold with the same strict comparison used in validation, followed by its 8-connected minimum-area filter. Filtering before thresholding does not define the same binary connected components. Finally, I would encode each mask with 1-based, column-major RLE.
- To reduce holdout overfitting, I would select thresholds and minimum areas on a tuning split and evaluate that frozen choice once on a separate audit split. I would also keep the search small and check whether the chosen settings remain useful across folds or seeds. I would track ForegroundDice, FalsePositiveScore and EmptyAccuracy separately. The all-empty baseline can obtain a high naive Dice simply because most pairs are empty, so that metric alone could reward thresholds that erase real defects.
- I would write a completely empty mask as a single space in mask_rle, rather than a missing value. Before submitting, I would check the CSV against sample_submission for the exact row count, columns, image/class IDs, ordering and valid RLE bounds. I would also run the whole notebook in the submission environment and check wall time and peak memory. A batchwise float32 running sum avoids storing every model/view prediction; I would leave time for decoding, component filtering, RLE and file I/O instead of spending the full limit on model passes.

Reproducibility

The notebook sources are notebooks/milestone-4.ipynb and notebooks/milestone-5.ipynb in the project repository. The separate Kaggle notebooks are 23f3000839-genai-milestone4-t32026 and 23f3000839-genai-milestone5-t32026. Saved CSV histories, split IDs, calculations and W&B verification records accompany the answers. No result in these exercises is presented as a new public leaderboard score.